

ПРИЛОЖЕНИЕ Д. РУКОВОДСТВО АДМИНИСТРАТОРА И ПРОГРАММИСТА

Техническое руководство по сопровождению, конфигурированию и развертыванию кода приложения «Glossa»

Настоящий документ предназначен для инженеров-программистов, системных администраторов и контент-менеджеров, осуществляющих техническое сопровождение, модификацию исходного кода и администрирование облачной инфраструктуры мобильного обучающего комплекса «Glossa».

1. Архитектура назначения и структура репозитория

Мобильное приложение разработано в рамках компонентного подхода на базе библиотеки **React Native** и набора инструментов **Expo SDK**. В качестве бэкенд-решения (Backend-as-a-Service) развернута инфраструктура **Google Firebase**. Перенос программного решения на сторонние рабочие станции предполагает понимание структуры файлового дерева проекта.

Основные каталоги и файлы конфигурации:

- `/src/components/` — библиотека переиспользуемых атомарных компонентов интерфейса (кнопки, текстовые обертки с поддержкой шрифтов, индикаторы прогресса, поля ввода).
- `/src/screens/` — модули экранов приложения (авторизация, регистрация, админ-панель, профиль, учебная сессия).
- `/src/firebase/config.js` — точка инициализации SDK Firebase, экспортирующая синглтоны сервисов авторизации (auth) и базы данных (db).
- `/src/utils/streakLogic.js` — изолированный алгоритмический модуль подсчета ежедневной активности пользователя.
- `package.json` — манифест проекта, фиксирующий версии зависимостей ядра (React, Expo) и сторонних пакетов.

2. Порядок развертывания рабочего окружения программиста

Для модификации кода, сборки тестовых версий и проведения отладки на локальной рабочей станции необходимо развернуть соответствующий инструментарий:

1. Установить платформу **Node.js** версии LTS (рекомендуется v18.x или v20.x). Менеджер пакетов `npm` поставляется в комплекте.
2. Установить распределенную систему контроля версий **Git** и редактор кода (например, Visual Studio Code).
3. Склонировать исходный код проекта или распаковать архив в рабочую директорию.

4. Открыть консоль в корневом каталоге и запустить команду для загрузки модулей ядра и внешних библиотек:

```
npm install
```

3. Настройка и связывание облачной инфраструктуры Firebase

Приложение жестко завязано на сервисы Firebase. При развертывании нового экземпляра системы (для другого заказчика или тестового окружения) необходимо перепривязать ключи в файле `/src/firebase/config.js`.

3.1. Регистрация проекта в консоли Firebase:

1. Перейти по адресу console.firebase.google.com, авторизоваться в аккаунте Google и нажать «Create a project».
2. Включить модуль **Authentication**, активировав способ входа «Email/Password».
3. Включить СУБД **Cloud Firestore**, выбрав создание базы данных в режиме «Production mode» и указав ближайший географический регион расположения дата-центра.
4. В настройках проекта (Project Settings) добавить новое веб-приложение («Add app» -> иконка `</>`) для генерации конфигурационного объекта JSON.

3.2. Локальное конфигурирование:

Полученные учетные данные необходимо вставить в структуру инициализации внутри проекта:

```
const firebaseConfig = {
  apiKey: "AIzaSyA1...",
  authDomain: "glossa-app.firebaseio.com",
  projectId: "glossa-app",
  storageBucket: "glossa-app.appspot.com",
  messagingSenderId: "1234567890",
  appId: "1:1234:web:abcd"
};
```

4. Администрирование базы данных Cloud Firestore

Администратору системы необходимо отслеживать и поддерживать структуру коллекций в актуальном состоянии. СУБД NoSQL является документоориентированной, однако логика кода приложения ожидает строго фиксированные поля.

4.1. Управление индексами и правилами безопасности

Для предотвращения несанкционированного изменения контента учебных курсов обычными пользователями, администратор обязан загрузить в консоль Firebase в раздел «Firestore Rules» следующие правила доступа:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }
    match /lessons/{lessonId} {
      allow read: if request.auth != null;
      allow write: if request.auth != null && get(/databases/{database}/documents/users/{request.auth.uid}).data.role == 'admin';
    }
    match /grammar_library/{itemId} {
      allow read: if request.auth != null;
      allow write: if request.auth != null && get(/databases/{database}/documents/users/{request.auth.uid}).data.role == 'admin';
    }
  }
}
```

4.2. Назначение прав Администратора пользователю

По умолчанию при регистрации через интерфейс мобильного приложения всем пользователям присваивается поле **"role": "user"**. Для перевода учетной записи в статус разработчика/модератора необходимо:

- Открыть консоль Firebase -> Cloud Firestore -> Коллекция **users**.
- Найти документ пользователя по его уникальному **UID** (значение можно скопировать из вкладки **Authentication**).
- Отредактировать значение поля **role**, изменив строковое значение с **"user"** на **"admin"**. Данные сохраняются мгновенно. После этого у пользователя в мобильном интерфейсе разблокируется доступ к экрану **AdminScreen.js**.

5. Компиляция и сборка финального дистрибутива приложения

Когда модификация исходного кода завершена, для публикации приложения в магазины или передачи заказчику необходимо собрать бинарные файлы (APK для Android, IPA для iOS). В экосистеме Expo сборка осуществляется через облачный инструмент **EAS Build (Expo Application Services)**.

Инструкция по сборке автономного файла:

1. Установить глобальный CLI-пакет EAS в систему: `npm install -g eas-cli`.
2. Выполнить авторизацию в облаке Ехро: `eas login` (требуется предварительно созданный аккаунт на expo.dev).
3. Инициализировать конфигурацию сборки в проекте: `eas build:configure`.
4. Для запуска сборки Android-приложения в формате APK (для прямой установки на устройства минуя Google Play) выполнить команду:

```
eas build --platform android --profile preview
```

5. Облачный сервер Ехро выполнит сборку проекта в изолированном контейнере и по завершении предоставит прямую ссылку на скачивание готового файла дистрибутива.

Критически важное предупреждение для разработчика: Никогда не передавайте файлы конфигурации `config.js` с открытыми ключами доступа в публичные репозитории GitHub/GitLab. Для защиты данных используйте механизм переменных окружения (файлы `.env`) совместно с пакетом `react-native-dotenv`.